

Bonus Project

CS-4240 – Compilers & Interpreters

David Folger, Devin Fromond, Rupesh Seetharaman

24 November, 2025

1. Backend Architecture and Algorithms

Our implementation begins by reading Tiger IR, building a CFG per function, and walks instructions to output MIPS32. The stack frame layout reserves space for params/locals, saves `$fp/$ra`, spills locals to the frame, and finally adds call/returns. Since this project is built from Project 2, we have three register allocation modes selectable via `run.sh`: **native** (always load/store from the stack), **greedy** (per-basic-block interference graph coloring using temporary variables), and **color** (global Chaitan-Briggs optimistic coloring). For **color**, we compute use/def sets, run iterative liveness across the CFG, build an interference graph, pick spill candidates with a degree/usage heuristic, and assign colors from `$s0-$s7`. By keeping spilled values on the stack, we reduce repeated memory loads across blocks without compromising correctness. The code generation is relatively linear: each IR opcode maps to a small MIPS sequence; arrays use base + offset addressing; calls handling up to four arguments in `$a0-$a3` and spill extras to the caller's stack. We map `geti/puti/putc/putc/getc` and other intrinsics directly to syscalls.

2. IR Program and Usage Instructions

Our custom tests within `custom_tests/custom_test.ir` computes a simple arithmetic sum: starting at 0, it adds 7 to the AC 2000 times, then prints the total of 14000. No inputs are required. The expected output is 14000.

Dependencies: Java, javac, bash

Build: `./build.sh`

Run: `./run.sh <input.ir> <out.s> [--native|--greedy|--color|-o|--flag]`

Example: `./run.sh custom_tests/custom_test.ir out.s -flag`

(flag defaults to optimized/color if unrecognized)

Generated MPIS can be executed with SPIM via `spim -keepstats -file out.s[< input]`.